Lab 01

OpenGL Programming Environment & Demo

Objectives

- To gain a good idea what OpenGL can do and what to expect of the rest of the trimester.
- To learn the programming environment for OpenGL programming.
- To have an overview of what an OpenGL program is.
- To experiment the basic functions in an OpenGL program.

Special Instructions

READ these lab instructions BEFORE coming to the lab!

Introduction

In this lab, you will be given a relatively big OpenGL program to play around with. This program would be used throughout the next few labs and you will be guided through in understanding the program as well as making changes of your own to the program. You will learn to write similar program throughout the trimester, for the time being, just try to play around with the program, try to make some changes and see the effect.

All programs in the lab materials throughout the whole trimester are written in Standard ISO/ANSI C++ and they are platform-independent for any platform with OpenGL and GLUT library installed. You may compile and run the programs in Microsoft Windows or the GNU/Linux environment.

Compiler

The official compiler for this course is the C++ compiler of **GCC** (GNU Compiler Collection). It is freely available for the GNU/Linux and Microsoft Windows platform. The GCC implementation under Microsoft Windows is called **MinGW** (http://www.mingw.org/). You may compile your program using the command-line commands of MinGW / GCC.

IDE

The official IDE for this course is the **Code::Blocks** IDE, this is a free C++ IDE, available for Windows as well as Linux. You can download it from http://www.codeblocks.org/.

Note: Codeblocks 17.12 (https://sourceforge.net/projects/codeblocks/files/Binaries/17.12/Windows/codeblocks-17.12mingw-setup.exe/download)

The IDE is actually an optional add-on GUI development environment; it still uses the GCC compiler (and also other compilers). This should already have been installed in the labs.

For your own personal use, you may choose to use other IDEs that you are comfortable with.

Programming Exercises (120 minutes)

Along with this `Lab01.pdf`, you will be provided the following files:
- `CGLabmain.cpp`
- `CGLabmain.hpp`
- `CGLab01.cpp`
- `CGLab01.hpp`
- `Lab01.cbp` (CodeBlocks project file for this lab)
- `glut32.dll` (for **d**ynamic **l**ink **l**ibrary)
- `OpenglSoftware.zip` (Files for OpenGL library)
- A folder called `data`, which contains the dragon model and some other models.

Program `CGLabmain.cpp` and `CGLabmain.hpp` are two files that will be used throughout all the labs.

`CGLabmain.cpp` will remain almost the same for Lab01 to Lab11, only two lines will need to be modified for each lab (unless you want to change some settings). As for Lab12, a modified version will be used. This program serves as the main program of which we will set up the OpenGL environment and call our lab-specific drawing/rendering programs (`CGLab01.hpp` and `CGLab01.cpp` for this lab) from this main program.

`CGLabmain.hpp` will remain the same for all the labs (i.e. Lab01 to Lab12), you will not need to modify this file, unless you wish to change something or add something for your own purposes.

`CGLab01.cpp` and its associated header file `CGLab01.hpp` are the programs that declared and defined the models to be drawn / rendered for this lab; these two files will be replaced for each of the labs.

Task 1 : Install OpenGL

Before you could program in OpenGL, you will need to install the necessary OpenGL file into your system.

If you go to `File->New->Project...` menu of CodeBlocks, you may find that there is already an "OpenGL project" template. However, if you use this template, you will need to know the tedious task of programming Win32 API to create the windows and setting up the windows environment etc. There is a simpler way to do this, which is by utilizing the GLUT library. The following will show you how to create OpenGL projects (with GLUT) in the CodeBlocks environment.

If you unzip the `OpenglSoftware.zip` file, you will see the files as in the table below:

	For	OpenGL Core Library	OpenGL Utility Library	OpenGL Utility Toolkit (GLUT)
Header Files	All Compilers	`gl.h`	`glu.h`	`glut.h`
Library Files	Code::Blocks / GCC / MinGW	`libopengl32.a`	`libglu32.a`	`libglut32.a`
DLL Files	All Compilers	`opengl32.dll`	`glu32.dll`	`glut32.dll`

OpenGL Core Library and OpenGL Utility Library are standard in all OpenGL implementation. GLUT is a platform-independent add-on to ease the task of managing windows environment. Your system may already have these files, in which case, you should always use the latest version, you should not replace those if the files already available.

1. Header Files

All header files should go to the "`include\GL`" folder of your compiler, thus:
- For CodeBlocks, it is "`C:\Program Files\CodeBlocks\MinGW\include\GL`"
Note: This assumes the compiler has been installed together with CodeBlocks.

Please **REPLACE** the existing files in `GL` folder with the provided `gl.h`, `glu.h` and `glut.h`.

2. DLL Files

All DLL files should go to folder "`C:\Windows\system32`"
Copy and paste the `glu32.dll, glut32.dll` and `opengl32.dll` to the above folder.

If the system does not allow the replacement, please click on "skip" button.

3. Library Files

All library files should go to the "`lib`" folder of your compiler, thus:
- For Code::Blocks, it is "`C:\Program Files\CodeBlocks\MinGW\lib`"
Note: This assumes the compiler has been installed together with CodeBlocks.

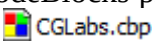
Copy and paste the `libglu32.a, libglut32.a` and `libopengl32.a` into the "`lib`" folder. Replace the existing files if there is any.

Take note that, you may have seen other files such as `opengl.lib`, `glu.lib`, `glut.lib`, `glu.dll`, `glut.dll` etc., these files are most likely for 16-dbits system, you should use the 32-bits version above if you program under Windows XP or Windows 7 environment.

Task 2 : Test the OpenGL Installation

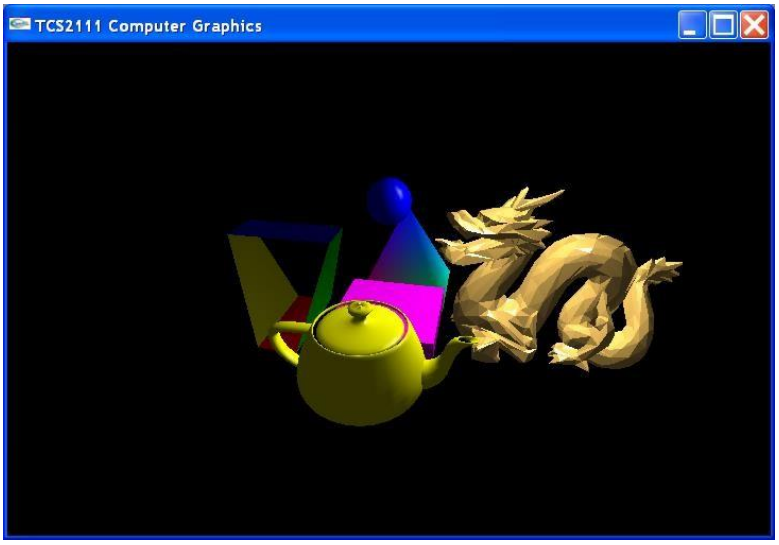
Make sure you have already copy the following files into the same folder:
- `CGLabmain.cpp`
- `CGLabmain.hpp`
- `CGLab01.cpp`
- `CGLab01.hpp`
- `Lab01.cbp` (CodeBlocks project file for this lab)
- `glut32.dll` (for **d**ynamic **l**ink **l**ibrary)

If you are using CodeBlocks, do the following:

1. Open the project file by selecting `File->Open…` menu and find the project file `Lab01.cbp` and open it. All CodeBlocks project files have the extension .CBP, and it is easy to spot by the 4-color icon… 

2. Check if the linker settings are correct. Go to menu "`Settings`", choose "`Compiler and debugger..`". Click on the `Linker Settings` tab, click on "`Add`" button and "`browse`" to the 3 library files – `libopengl32.a`, `libglu32.a`, and `libglut32.a` in the MinGW "`lib`" folder. [When you run the program, if it shows the error of "`Invalid Compiler`", please go to "`Settings`", "`Compiler and debugger`", choose "`Toolchain executables`", under the Compiler's installation directory, you should see the path `C:\Program Files\CodeBlocks\MinGW`, if not, type this path in the text box **OR** simply click on "Auto-detect" button and this will automatically detect a compatible compiler for you.] You only need to do this for the **first time** when you install the OpenGL Software into your PC.
3. If your machine has multiple MinGW compilers installed (probably due to other IDE packages or MinGW installations), you may encounter a compilation error that indicates searching the wrong `include` directory for files. In this case, you will have to add the absolute path, `C:\Program Files\CodeBlocks\MinGW\include` (do NOT add as relative path), to the `Search directories->Compiler` tab under `Build options…` If your machine only has a single installation, you do not need to add any directory.
4. Build the project by one of the methods below:
   - Right-click the mouse at the project name and select `Build`
   - Select `Build->Build` menu
   - Press the shortcut key `Ctrl-F9`
   - Click at the `Build` icon at the toolbar 
5. If there are compilation errors, then go back to Task 1 above and make sure OpenGL is properly installed. If you still face problems after copying all the necessary files into the correct folders, then you may need to copy and replace some of the files as in Task 1with different version of OpenGL software. This may due to incompatibility problems of the various files, HOWEVER, please consult the tutors before you do so and backup the files that you are replacing.
6. If there are no compilation errors, then you may run the program by one of the methods below:
   - Select `Build->Run` menu
   - Press `Ctrl-F10`
   - Click at the `Run` icon at the toolbar 
7. Alternative, you may press `F9` to Build and Run at once.

If everything goes well, you should see an output window with the following objects:



There should also be another output window that displays some text about the various key commands that you can use to move/change/manipulate settings in the world. In the next task, we will explore that further.

Task 3 : Explore the output

Try pressing the keys such as ‘A’, ‘D’, ‘S’, ‘W’, ‘Q’, ‘E’ etc. as stated in the program output.

You can also try to move the mouse while pressing the left or right mouse buttons. What effect did you see?

Try pressing F1, F2 or F3 keys and see what you will see. Discuss what you observed.

Do you notice the different colours shading of the various objects?

Try to look at the pyramid from the bottom, what did you see? If you can not see anything from the bottom of the pyramid, do not panic! We will get to the reasons and rectify it in next lab.

Task 4 : Experiment with…
a) Some world/viewing parameters

Part of the `myDataInit()` function of `CGLabmain.cpp` is as below:

```
void myDataInit()
{
 window.title = "TCG6223 Computer Graphics";
 window.posX = 100;
 window.posY = 100;
 window.width  = 800;
 window.height = 500;

 world.rotateX  = 0.0;
 world.rotateY  = 0.0;
 world.rotateZ  = 0.0;
 world.posX     = 0.0;
 world.posY     = 0.0;
 world.posZ     = 0.0;
 world.scaleX   = 1.0;
 world.scaleY   = 1.0;
 world.scaleZ   = 1.0;

 viewer.eyeX    = 0.0;
 viewer.eyeY    = 0.0;
 viewer.eyeZ    = 40.0;
 viewer.centerX = 0.0;
 viewer.centerY = 0.0;
 viewer.centerZ = 0.0;
 viewer.upX     = 0.0;
 viewer.upY     = 1.0;
 viewer.upZ     = 0.0;
 viewer.zNear   = 0.1;
 viewer.zFar    = 500.0;
 viewer.fieldOfView = 60.0;
 viewer.aspectRatio = static_cast<GLdouble> (window.width) / window.height;

 setting.posInc   = 1.0;
 setting.angleInc = 2.0;
 setting.mouseX   = 0;
 setting.mouseY   = 0;

 setting.mouseRightMode = false;
 setting.mouseLeftMode = false;

 setting.shadingMode = true;
}
```

Try to modify those values in this function and run the program again, what are the effects? Can you understand how your changes affect the output?

Do not try to change too many parameters at one time; you will get very confused with the output. Instead, change **ONE** setting at a time. Once you understood how that particular setting affects the output, then you can explore the other parameters.

Here's some interesting suggestions for you to try:

i.
```
world.rotateX = 0.0;
world.rotateY = 45.0;
world.rotateZ = 45.0;
```

ii.
```
world.posX    =   0.0;
world.posY    =   0.0;
world.posZ    = -10.0;
```

iii.
```
world.scaleX  = 1.0;
world.scaleY  = 2.0;
world.scaleZ  = 0.5;
```

iv.
```
viewer.eyeX   =   0.0;
viewer.eyeY   = -20.0;
viewer.eyeZ   =  40.0;
```

v.
```
viewer.centerX =   0.0;
viewer.centerY = -20.0;
viewer.centerZ =   0.0;
```

vi.
```
viewer.upX    = 1.0;
viewer.upY    = 0.0;
viewer.upZ    = 0.0;
```

vii.
```
viewer.zNear  = 38.5;
viewer.zFar   = 500.0;
```
(Try zoom in and out and rotate the objects around and see)

viii.
```
viewer.fieldOfView = 140.0;
```
(Try zoom in and out and rotate the objects around and see)

ix.
```
viewer.fieldOfView = 20.0;
```
(Try zoom in and out and rotate the objects around and see)

b) Different colours

All the colours that are in `CGLab01.cpp` are specified by statement such as this:

```
glColor3f(1.0f, 1.0f, 0.0f);
```

The first, second and third numbers represent the intensity values of red, green and blue component respectively. All intensity values are from 0.0 to 1.0.

Modify the color information, and see how the outputs are affected.

c) Loading different models

From the `init()` function of `MyVirtualWorld` class in `CGLab01.hpp`, try to load other models in the `data` folder provided. Try out the high-polygons dragon model (some PC may be too slow to handle that), or rose model. Take note that there is a scaling factor as the second argument of the `load()` function calls within `init()` function.

Task 5 : Create A New OpenGL Project File

Earlier in Task 2, you were given the `Lab01.cbp` project file. In the following, we will go through the steps to create our own OpenGL project files.

If you are using CodeBlocks IDE (version 17.12), do the following:

- You may choose to close the existing project by selecting `File->Close project`. However, one advantage of CodeBlocks is that you can have many projects opened in one workspace, but you need to specify which is you current active project.
- Select `File->New->Project...` and choose `Console Application` template (do not choose the built-in `OpenGL Application` template as it is based on Win32 API instead of GLUT) and press the `Go` button to initiate the wizard.
- The Wizard is quite easy to follow. Specify the language that we want to use, which is C++. Next, type in your new project file name, and let's call it `MyCGLabs`. This will cause a file called `MyCGLabs.cbp` being created. Also, select the location of where you want to save the project file (ideally in the same folder with `CGLabmain.cpp` and the other files).
- Next, the compiler should already be selected as "GNU GCC Compiler" unless you have not properly installed or setup the correct compiler for CodeBlocks. "Debug" and "Release" configurations will be checked by default. You can just leave it as it is for the moment. We will not go into the details on what each of these configurations represent. Press Finish and your project will be set up.
- Now, you may notice inside the left panel under the Default workspace, a project named "MyCGLabs" has been created. If you wish, you may rename this by right-clicking at it, choose `Properties` and change the title.
- If you expand the `MyCGLabs` project tree, you will see a file called `main.cpp` has been created for you under "Sources". Normally, you can straight away code in this file (just like what you have learnt before). However, since we already have all the `.cpp` and `.hpp` files as provided above, we will delete this `main.cpp` file from the project by right-click at this file and select `Remove file from this project`.
- Add all the required `.cpp` and `.hpp` files into the project by right-clicking at the `MyCGLabs` project and select `Add files...` menu, then browse and add the `CGLabmain.cpp`, `CGLabmain.hpp`, `CGLab01.cpp` and `CGLab01.hpp` files into the project. You can select all of them by holding on to the Ctrl button when selecting.
- Now, you need to include the necessary OpenGL library files into the project. To do so, right-click at `MyCGLabs` project and select the `Build options` menu, then select the `Linker settings` tab and press `Add` button. Then you should select the 3 files – `libopengl32.a`, `libglu32.a`, and `libglut32.a` (from the folder mentioned in Task 1) to be added into the project. To complete the process, answer "No" to the "Keep this as a relative path?" question. Note: You may go to "`Settings`", "`Compiler and debugger`", ... same with the instruction in Task 2 for linking the OpenGL libraries.

- If your machine has multiple MinGW compilers installed, you will have to add `C:\Program Files\CodeBlocks\MinGW\include` (do NOT keep the location as a relative path) to the list of directories the compiler will search for additional files, which is in the `Search directories->Compiler` tab under `Build options…` This step is normally needed when you have other compilers or similar MinGW compilers installed elsewhere in the machine that may cause conflicting directories in the system path. If you machine only has a single installation, you need not add any directory.
- Now that your `MyCGLabs.cbp` project file has been created, you can compile, build and run your project just like in Task 2 above.

(Note that steps to create an OpenGL project file may be different in other IDEs, but these are simply basic things you should know when it comes to creating projects)
If you have problem following the above steps please pay close attention to your tutor's explanation and demo!

Task 6 : Explore and Have Fun

If you have done all the tasks above and you still have time, then try to read through the programs and see if you can understand any code inside, don't worry if you can't, we will dissect the code part by part in the later labs throughout the trimester.

Be adventurous! Be creative! ... and HAVE FUN !!

Extra Exercises

VERY IMPORTANT

BEFORE coming to the next lab, you MUST:

1. COMPLETE all the exercises in this lab!

2. DO the extra exercises below (if there are any)!

3. READ the lab instructions!

4. PREPARE files for next lab!

OpenGL programming is best learned by examples, and this course intends to do just that. Try to refer to websites, books etc. and see if you can understand the programs given in this lab and make intelligent changes to it and see if the effects are as, you expected them to be.

Please try and attempt **Task 1A** and **1B** of the next lab, **Lab02** on your own before the next lab session.

** END OF LAB **